DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 11:
Parallel Matrix Multiplication

What is dense linear algebra?

Not just matrix multiply!

- Linear systems: $Ax = b$
- Least squares: choose x to minimize $\|Ax - b\|_2^2$
  - Overdetermined or underdetermined; Unconstrained, constrained, weighted
- Eigenvalues: $Ax = \lambda x$ (or $Ax = \lambda Bx$)
- Matrix factorization
  - $A = LU$, $A = LL^T$
  - $A = QR$
  - $A = V\Lambda V^{-1}$, $A = QTQ^T$
  - $A = U\Sigma V^T$

A brief history of (Dense) Linear Algebra software

BLAS 1 were invented (1973–1977)

- Standard library of 15 ops (mostly) on vectors
  - Up to four versions of each: S/D/C/Z,), 46 routines, 3300 LOC
  - Example: DAXPY
    - Double precision (real)
    - Computes $Ax + y$
  - Goals
    - Common “pattern” to ease programming, readability
    - Robustness, via careful coding (avoiding over/underflow)
    - Portability + Efficiency via machine specific implementations
  - BLAS 1 == $O(n^1)$ ops on $O(n^1)$ data
  - Used in libraries like LINPACK
  - Consider AXPY ($y = \alpha x + y$): $2n$ flops on $3n$ read/writes; so computational intensity is just $2/3$

A brief history of (Dense) Linear Algebra software

BLAS 2 were invented (1984–1986)

- Standard library of 25 ops (mostly) on matrix/vector pairs
  - Different data types and matrix types
  - Example: DGEMV
    - Double precision
    - GEneral matrix
    - Matrix-Vector product
- Goals
  - BLAS1 insufficient
  - BLAS2 for better vectorization (when vector machines roamed)
- BLAS2 == $O(n^2)$ ops on $O(n^2)$ data
- Computational intensity of AXPY is 2; still too low to run near peak speed

A brief history of (Dense) Linear Algebra software

BLAS 3 (1987–1988)

- Standard library of 9 ops (mostly) on matrix/matrix pairs
- Different data types and matrix types
- Example: DGEMM
  - Double precision
  - GEneral matrix
  - Matrix-Matrix product
  - Ex.: $C = \alpha AB + \beta C, C = \alpha AA^T + \beta C, B = T^{-1}B$
- BLAS3 == $O(n^3)$ ops on $O(n^2)$ data
- So computational intensity AXPY $(2n^3)/(4n^2) = n/2$
  - Big for efficient cache utilization
- Source: 142 routines, 31K LOC, Testing: 28K LOC
- Part of standard math libraries (eg AMD ACML, Intel MKL)
- `http://www.netlib.org/blas`

LAPACK: “Linear Algebra PACKage”

LAPACK (1989–now): `http://www.netlib.org/lapack`

- Uses BLAS-3
  - Parallel to the extent BLAS are parallel (on SMP)
  - Linear systems and least squares are nearly 100% BLAS 3
  - Eigenproblems, SVD — only about 50% BLAS 3
- Careful error bounds on everything
- Lots of variants depending on A’s structures

ScaLAPACK: “Scalable LAPACK”

ScaLAPACK (1995–present):
`http://www.netlib.org/scalapack`

- For distributed memory - uses MPI implementations
- More complex data structures, algorithms than LAPACK
- Only a small subset of LAPACK functionality

PLASMA and MAGMA

PLASMA and MAGMA (2008–now):

- Parallel LA Software for Multicore Architectures
  - Target: Shared memory multiprocessors
  - Stacks on LAPACK/BLAS interfaces
  - Tile algorithms, tile data layout, dynamic scheduling
  - Other algorithmic ideas, too (randomization, etc)
- Matrix Algebra for GPU and Multicore Architectures
  - Target: CUDA, OpenCL, Xeon Phi
  - Still stacks (e.g. on CUDA BLAS)
  - Again: tile algorithms + data, dynamic scheduling
  - Mixed precision algorithms (+ iterative refinement)
- Dist memory: DPLASMA

LAPACK routine types

- Linear systems (general, symmetric, SPD)
- Least squares (overdetermined, underdetermined, constrained, weighted)
- Symmetric eigenvalues and vectors
  - Standard: $Ax = \lambda x$
  - Generalized: $Ax = \lambda Bx$
- Nonsymmetric eigenproblems
  - Schur form: $A = QTQ^T$
  - Eigenvalues/vectors
  - Invariant subspaces
  - Generalized variants
- SVD (standard/generalized)
- Different interfaces
  - Simple drivers
  - Expert drivers with error bounds, extra precision, etc
  - Low-level routines

Communication Lower Bounds on Matmul

- Assume n^3 algorithm
- Reminder: Sequential case, with fast memory of size M
  - Lower bound on #words moved to/from slow memory $O(n^3/M^{1/2})$
  - Attained using blocked algorithms:
    - $4n^3/b$ reads/writes altogether
    - Minimized when $3b^2 = M$
- Parallel case on p processors:
  - Let M be memory per processor; assume load balanced, then $M = 3n^2/p$ (one copy of each matrix)
  - Number of flops done per processor = n^3/p
  - Lower bound on #words moved = $O(n^3/(pM^{1/2}))$
  - Attained by SUMMA (next)

Matrix vector product

Simple $y = Ax$ involves two indices

$$y_i = \sum_j A_{ij} x_j$$

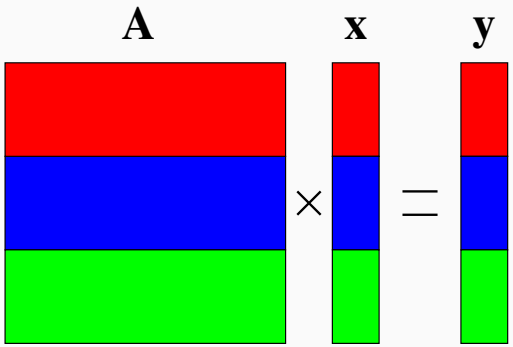
Can organize around either one:

```
% Row-oriented
for i = 1:n
  y(i) = A(i,:)*x;
end

% Col-oriented
y = 0;
for j = 1:n
  y = y + A(:,j)*x(j);
end
```

... or deal with index space in other ways!

Parallel matvec: 1D row-blocked



For each processor i, broadcast $x(i)$ then compute
$y(i) = A(i) * x$

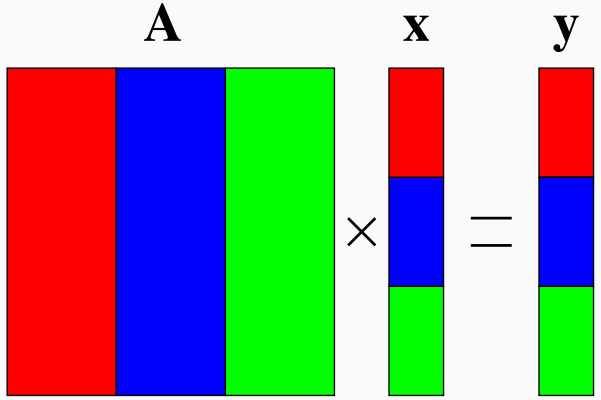
$$\text{On P0:} \quad A_{00}x_0 + A_{01}x_1 + A_{02}x_2 = y_0$$
$$\text{On P1:} \quad A_{10}x_0 + A_{11}x_1 + A_{12}x_2 = y_1$$
$$\text{On P2:} \quad A_{20}x_0 + A_{21}x_1 + A_{22}x_2 = y_2$$

Algorithm uses the formula $y(i) = A(i) * x = \Sigma_j A(i,j) * x(j)$

- $A(i)$ refers to the n by n/p block row that processor i owns
- $x(i)$ and $y(i)$ similarly refer to segments of x, y owned by i

Parallel matvec: 1D column-blocked

A column layout of the matrix eliminates the broadcast of x but adds a reduction to update the destination y:



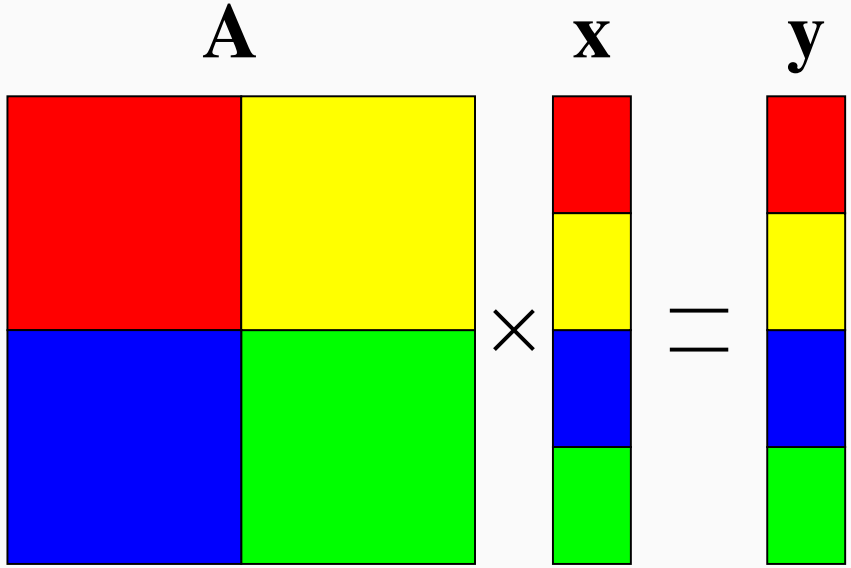
Independently compute

$$z^{(0)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_0 \quad z^{(1)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_1 \quad z^{(2)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_2$$

and perform reduction: $y = z^{(0)} + z^{(1)} + z^{(2)}$.

Parallel matvec: 2D blocked

A 2D blocked layout uses a broadcast and reduction, both on a subset of processors (sqrt(p) for square processor grid)



- Involves broadcast *and* reduction
- ... but with subsets of processors

Parallel matvec: 2D blocked

Broadcast x_0, x_1 to local copies x_0, x_1 at P0 and P2

Broadcast x_2, x_3 to local copies x_2, x_3 at P1 and P3

In parallel, compute

$$\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} z_0^{(0)} \\ z_1^{(0)} \end{bmatrix} \qquad \begin{bmatrix} A_{02} & A_{03} \\ A_{12} & A_{13} \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} z_0^{(1)} \\ z_1^{(1)} \end{bmatrix}$$

$$\begin{bmatrix} A_{20} & A_{21} \\ A_{30} & A_{31} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix} \qquad \begin{bmatrix} A_{20} & A_{21} \\ A_{30} & A_{31} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix}$$

Reduce across rows:

$$\begin{bmatrix} y_0 \\ y_1 \end{bmatrix} = \begin{bmatrix} z_0^{(0)} \\ z_1^{(0)} \end{bmatrix} + \begin{bmatrix} z_0^{(1)} \\ z_1^{(1)} \end{bmatrix} \qquad \begin{bmatrix} y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} z_2^{(2)} \\ z_3^{(2)} \end{bmatrix} + \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix}$$

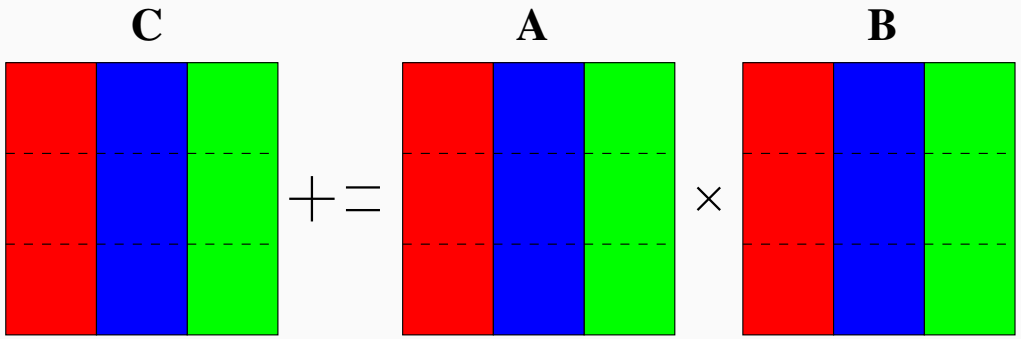
Parallel Matrix Multiplication

The performance depends on several factors:

- The interconnection network. Networks with higher connectivity, like the hypercube and mesh, permit faster algorithms than networks like rings and busses.
- The data layout, or where A, B and C are stored on the processors.
- The algorithm.
  - Basic operation: $C = C + AB$
  - Computation: $2n^3$ flops
  - Goal: $2n^3/p$ flops per processor, minimal communication

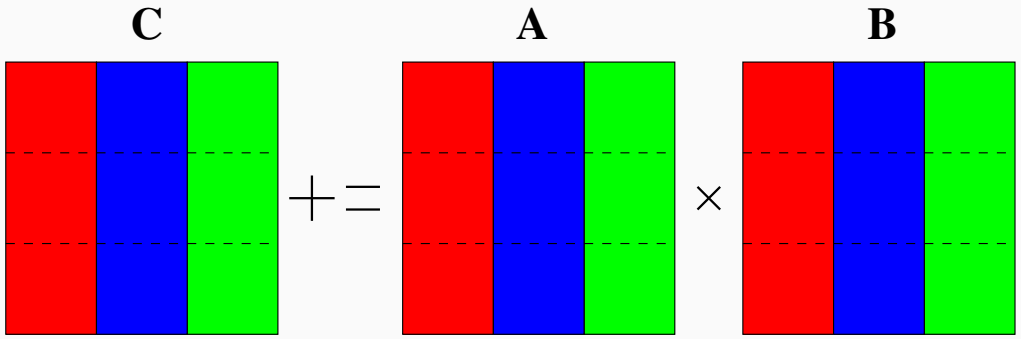
Use of performance models for algorithm design:
Message Time = latency + #words * time-per-word = $\alpha + n * \beta$

1D Column Blocked Layout



- $A(i)$ refers to the n by n/p block column that processor i owns
- Processor i owns $A(i)$, $B(i)$, $C(i)$
- $C(i)$ depends on *all* of A, but only $B(i)$, $C(i) = C(i) + A * B(i)$
- Only A needs to be communicated; how do we communicate pieces of A?

1D layout on bus (no broadcast)

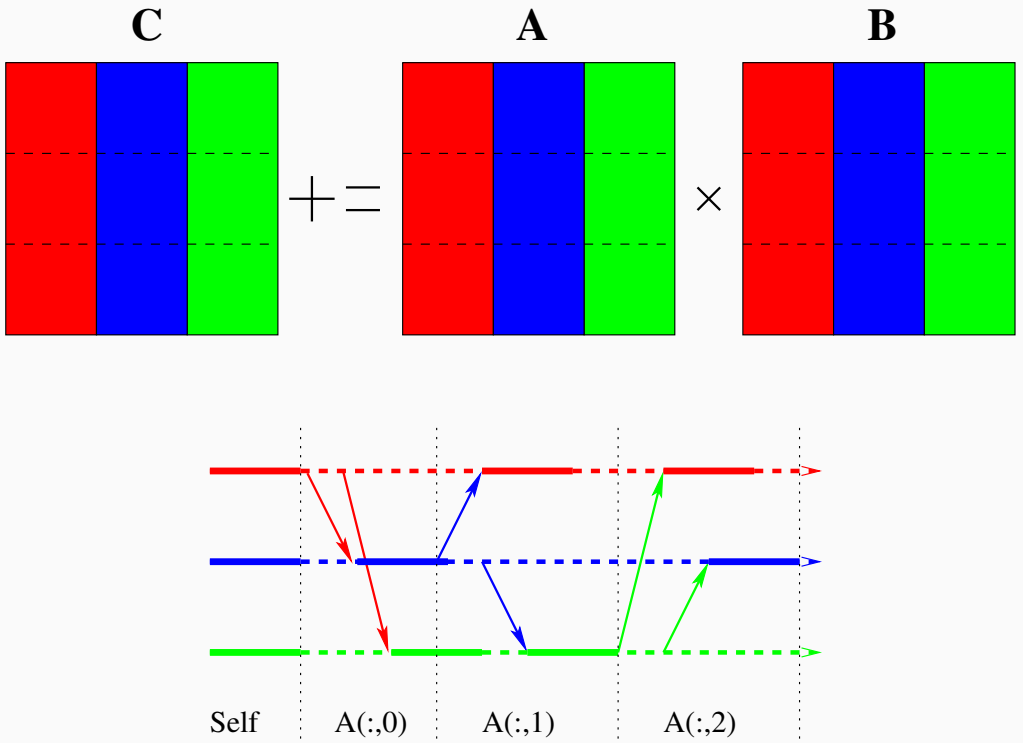


- Everyone computes local contributions first
- P0 sends $A(0)$ to each processor j in turn; processor j receives, computes $A(0)B(0,j)$
- P1 sends $A(1)$ to each processor j in turn; processor j receives, computes $A(1)B(1,j)$
- P2 sends $A(2)$ to each processor j in turn; processor j receives, computes $A(2)B(2,j)$

$B(i,j)$ is the n/p by n/p sublock of $B(i)$ in rows $j * n/p$ through $(j+1) * n/p - 1$

1D layout on bus (no broadcast)

First consider a bus-connected machine without broadcast: only one pair of processors can communicate at a time (ethernet)



Only 1 pair of processors (*i* and *j*) are active on any iteration, and of those, only *i* is doing computation; no overlapping

1D layout on bus (no broadcast)

Algorithm uses the formula

$C(i) = C(i) + AB(i) = C(i) + \Sigma_j A(j) * B(j, i)$

```
1  C(myproc) += A(myproc)*B(myproc,myproc)
2  for i = 0:p-1
3    for j = 0:p-1, except i
4      if (myproc == i)
5        send A(i) to processor j
6      if (myproc == j)
7        receive A(i) from i
8        C(myproc) += A(i)*B(i,myproc)
9      barrier
```

Performance model: 1D layout on bus (no broadcast)

Only 1 pair of processors (i and j) are active on any iteration, and of those, only i is doing computation; no overlapping communications, so in a simple $\alpha - \beta$ model:

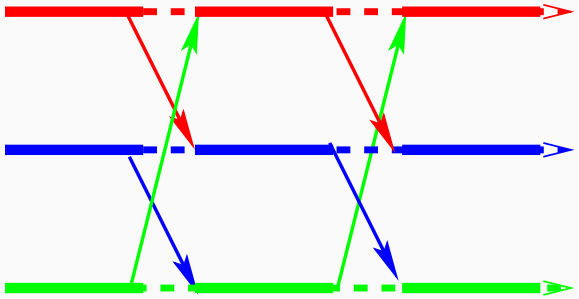
- Computations: $2n(n/p)^2 = 2n^3/p^2$
- Messages: $p(p-1)$
- Data: each message involves n^2/p words
- Communication cost: $p(p-1)\alpha + (p-1)n^2\beta$

Running time $\approx 2n^3 + p^2\alpha + pn^2\beta$

This is worse than the serial time and grows with p. Useful only for big problem that doesn't fit on one processor.

1D layout on a Processor Ring

Pairs of adjacent processors can communicate simultaneously.



- Every process i can send data to $i + 1$ simultaneously
- Pass slices of A around the ring until everyone sees the whole matrix ($p - 1$ phases).

```
1  tmp = A(myproc)
2  C(myproc) += tmp*B(myproc,myproc)
3  for j = 1 to p-1
4    sendrecv tmp to myproc+1 mod p,
5             from myproc-1 mod p
6    C(myproc) += tmp*B(myproc-j mod p, myproc)
```

Performance model: 1D layout on ring

In a simple $\alpha - \beta$ model, at each processor:

- $p - 1$ message sends (and simultaneous receives)
- Each message involves n^2/p data
- Communication cost: $(p-1)\alpha + (1-1/p)n^2\beta$

Running time $\approx 2n^3/p + 2p\alpha + 2n^2\beta$

Parallel efficiency grows to 1 as n/p increases (or as α, β get smaller)

Need to try 2D Matrix layout

Outer product algorithm

Serial: Recall outer product organization:

```
1  for k = 0:s-1
2    C += A(:,k)*B(k,:);
3  end
```

Parallel: Assume p processors, block $P^{1/2} \times P^{1/2}$ matrices.

For a 2×2 example:

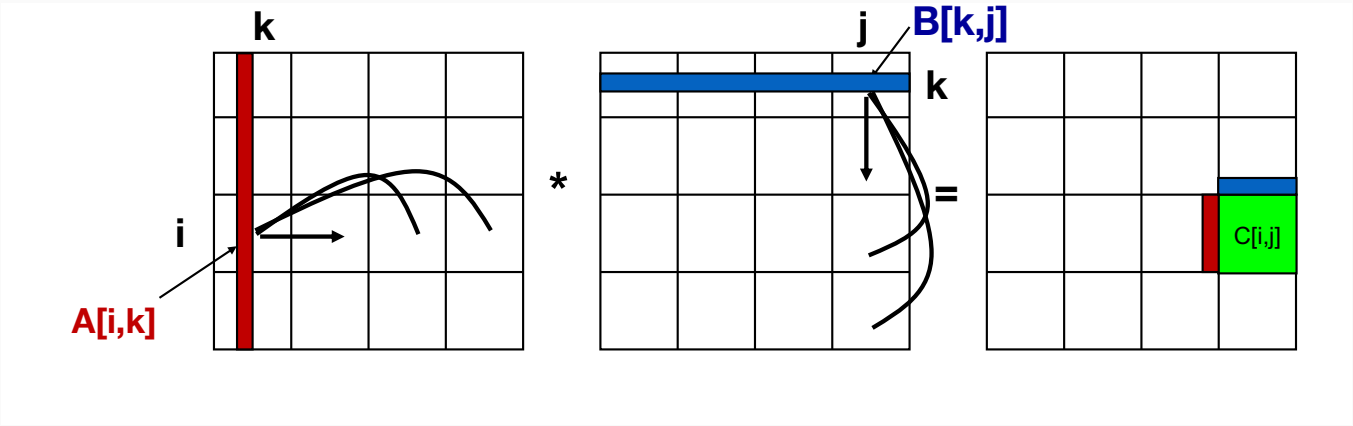
$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00}B_{00} & A_{00}B_{01} \\ A_{10}B_{00} & A_{10}B_{01} \end{bmatrix} + \begin{bmatrix} A_{01}B_{10} & A_{01}B_{11} \\ A_{11}B_{10} & A_{11}B_{11} \end{bmatrix}$$

- Processor for each $(i, j) \implies$ parallel work for each k!
- Note everyone in row i uses $A(i, k)$ at once,
  and everyone in row j uses $B(k, j)$ at once.

Parallel Matrix Multiply

SUMMA = Scalable Universal Matrix Multiply

Used in practice in PBLAS = Parallel BLAS



- $C[i,j]$ is $n/P^{1/2} \times n/P^{1/2}$ submatrix of C on processor P_{ij}
- $A[i,k]$ is $n/P^{1/2} \times b$ submatrix of A
- $B[k,j]$ is $b \times n/P^{1/2}$ submatrix of B
- $C[i,j] = C[i,j] + \Sigma_k A[i,k] * B[k,j]$
- Summation over submatrices
- Need not be square processor grid

Parallel outer product (SUMMA)

```
1  for k=0 to n/b-1
2    for all i = 1 to P^(1/2)
3      owner of A[i,k] broadcasts it to whole processor row
4      (using binary tree)
5  // #words  = log P^(1/2)*b*n/P^(1/2), #messages = log P^(1/2)
6    for all j = 1 to  P^(1/2)
7      owner of B[k,j] broadcasts it to whole processor column
8      (using binary tree)
9  // same #words and #messages
10       Receive A[i,k] into Acol
11       Receive B[k,j] into Brow
12       C_myproc = C_myproc + Acol * Brow
13 // #flops = 2n^2*b/P
```

Parallel outer product (SUMMA)

If we have tree along each row/column, then

- $\log(P^{1/2})$ messages per broadcast
- $\alpha + \beta n^2/P$ per message
- $2\log(P^{1/2})(\alpha P^{1/2} + \beta n^2/P^{1/2})$ total communication

Recall

$$\text{Speedup} := t_{\text{serial}}/t_{\text{parallel}}$$

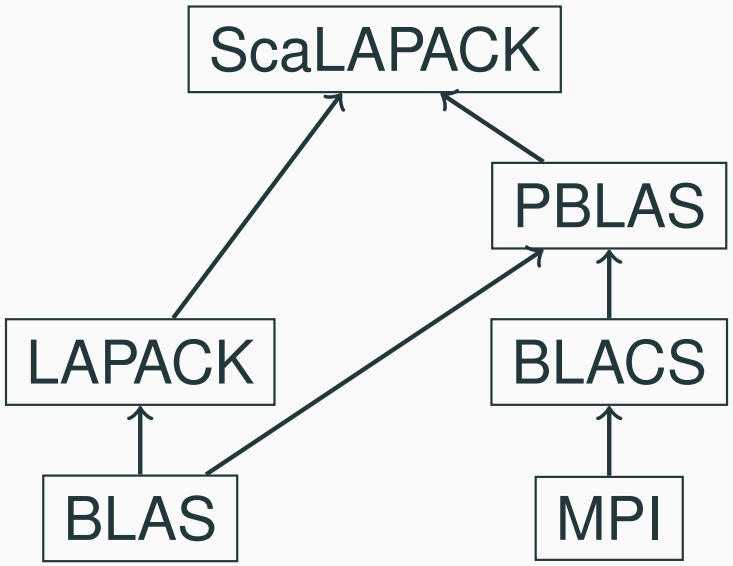
$$\text{Efficiency} := \text{Speedup}/p$$

Assuming no overlap of communication and computation, efficiencies are

$$\begin{array}{ll} \text{1D layout} & \left(1 + O\left(\frac{p}{n}\right)\right)^{-1} \\ \text{SUMMA} & \left(1 + O\left(\frac{\sqrt{p}\log p}{n}\right)\right)^{-1} \end{array}$$

Reminder: Why matrix multiply?

Build fast serial linear algebra (LAPACK) on top of BLAS 3.



ScaLAPACK builds additional layers on same idea.

Summary: Parallel Matrix Multiply

- **1D Layout**:
  - Bus without broadcast: slower than serial
  - Nearest neighbor communication on a ring (or bus with broadcast):

$$\text{Parallel Efficiency} = 1/(1 + O(p/n))$$

- **2D Layout**: SUMMA
  - Very General:

$$\text{Total Time} = 2 * n^3/p + \alpha * \log p * n/b + \beta * \log p * n^2/p^{1/2}$$

$$\text{Parallel Efficiency} = 1/(1+O(\alpha*\log p*p/(b*n^2)+\beta*\log p*p^{1/2}/n)$$

  - b small => less memory, lower efficiency
  - b large => more memory, high efficiency
  - Used in practice (PBLAS)